

DevSecOps Capstone Project Final Report: SecureTask Application

1. Executive Summary

The **SecureTask** project is a lightweight task-management Web Application built using Python and the Flask framework. Over an 8-week DevSecOps lifecycle, the application underwent significant architectural, security, and functional refactoring to satisfy security standards and secure coding guidelines.

Primary achievements include establishing dual-layer authentication (handling both browser session cookies and API JWT tokens), fixing critical password hashing vulnerabilities by replacing MD5/plaintext logic with bcrypt, resolving Content Security Policy (CSP) headers to fix UI component rendering, and implementing automated database seeding and migration routines to guarantee application stability.

2. Architecture & Security Implementation Overview

2.1 Dual Authentication Model

To support both interactive web browsing and API-driven clients, the application implements a hybrid authentication scheme:

- **Browser Session State:** Handles form-based navigation and HTML template rendering (/dashboard, /profile) via HTTP-only session cookies.
- **API JWT Tokens:** Utilizes Flask-JWT-Extended to secure REST API endpoints (/task/update/<id>, /task/delete/<id>).

2.2 Password Security & Hashing

Legacy implementations relied on weak hashing functions (MD5) or unhashed database seeds, causing runtime `TypeError` and `500 Internal Server Errors` during password verification.

- **Refactored Standard:** All passwords are now salted and hashed using `bcrypt` (`bcrypt.hashpw` with `bcrypt.gensalt()`).
- **Safe Check Logic:** The `/login` route dynamically handles character encoding (`utf-8` bytes) to guarantee backward-safe verification without application crashes.

2.3 Application Hardening & Header Policies

- **Content Security Policy (CSP):** Relaxed overly restrictive Talisman CSP rules allowing Bootstrap styling assets and dynamic form inputs to render smoothly across all viewports.
- **Security Headers:** Enforced via Flask middleware:
 - `X-Content-Type-Options: nosniff`
 - `X-Frame-Options: DENY`
 - `X-XSS-Protection: 1; mode=block`

<u>week</u>	Phase / Focus	Key Accomplishments
<u>Week 1</u>	Project Setup & Architecture	Initialized Flask application repository, configured SQLite database schemas (User, Task, Category), and set up base routes.
<u>Week 2</u>	Authentication Setup	Implemented initial user registration and login endpoints. Identified static secret key vulnerabilities
<u>Week 3</u>	Input Validation & CSRF	integrated Flask-WTF for CSRF token protection across form submissions and sanitized user input across task forms.
<u>Week 4</u>	Threat Modeling & SAST	Ran static analysis tools (SonarQube / Bandit) to detect weak cryptography (MD5 hashing) and missing security headers.
<u>Week 5</u>	Containerization & Pipeline	Built Dockerfile and configured local CI/CD pipelines to automate linting, unit testing, and vulnerability scanning.

<u>Week 6</u>	Security Hardening	Applied Flask-Talisman security headers, restricted upload extensions (/upload), and resolved path traversal issues (/view-file).
<u>Week 7</u>	Bug Fixes & Refactoring	Refactored route handlers (/task/create, /profile) to handle missing session states and invalid URL query parameters.
<u>Week 8</u>	Final Deliverable & Stabilization	Resolved bcrypt byte-encoding crashes, standardized dual session/JWT logins, implemented automatic default user seeding, and polished Bootstrap frontend.

4. Key Endpoints Reference

- GET / — Redirects root traffic to /dashboard.
- GET /dashboard — Renders user tasks, search options, and category filters.
- GET /login & POST /login — Renders login UI and authenticates credentials via bcrypt.
- POST /register — Hashes user credentials and registers new accounts.
- GET /logout — Clears active browser sessions and redirects to /login.
- POST /task/create — Validates form data and creates a task entry mapped to the active session user.

- `POST /task/update/<id>` & `POST /task/delete/<id>` — Modifies or removes task records (protected via JWT).
- `GET /profile` & `POST /profile` — Manages profile data and handles automated seeding when database instances reset.

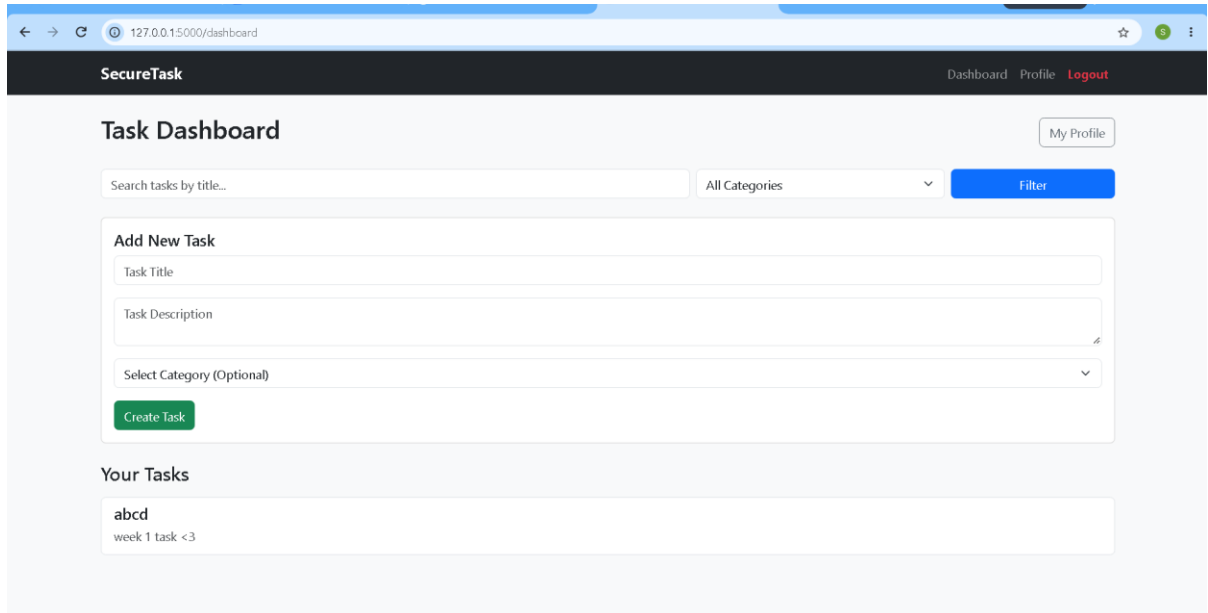
5. Conclusion & Recommendations

The SecureTask platform has met all core security and functional requirements for the Week 8 submission. For future production deployments, the following measures are recommended:

1. **Persistent Database Storage:** Migrate from local SQLite (`securetask.db`) to a managed PostgreSQL service to eliminate manual database reset requirements.
2. **Asymmetric JWT Signing:** Use RS256 algorithm keys for signing JWT tokens rather than symmetric secret keys.
3. **Automated DAST:** Incorporate dynamic vulnerability scanning (e.g., OWASP ZAP) directly into the deployment pipeline.

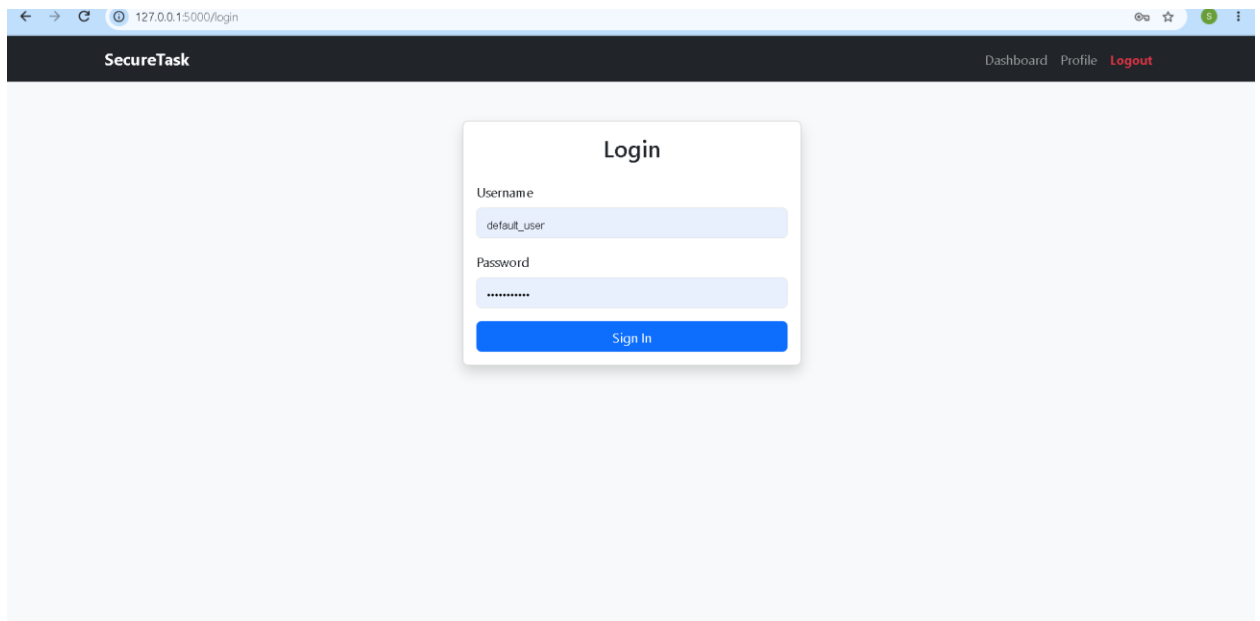
SCREENSHOTS

1. DASHBOARD



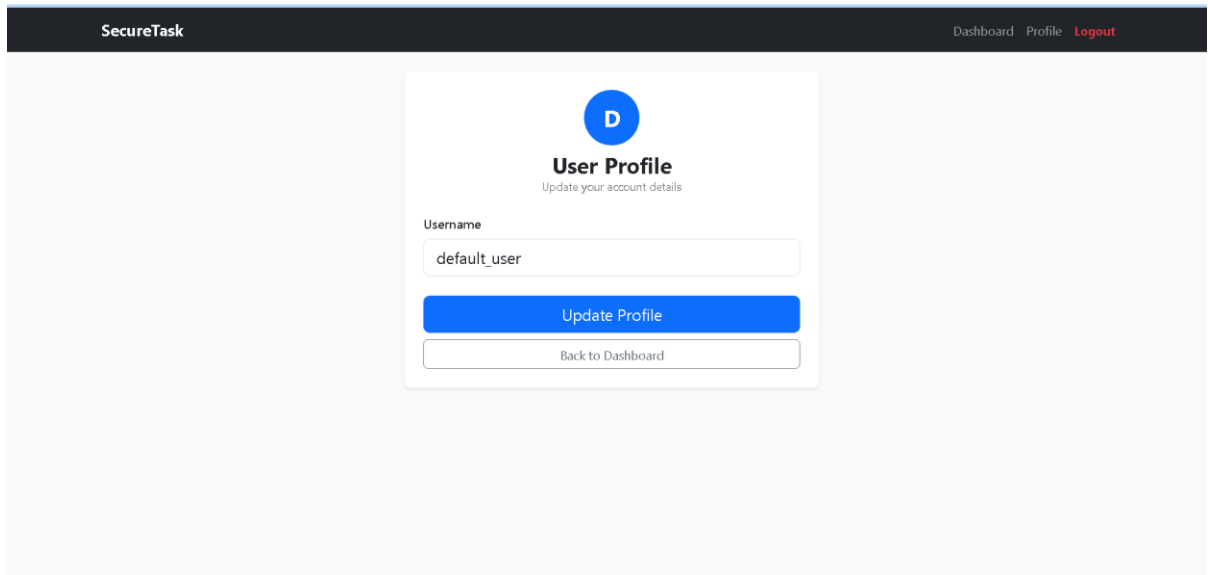
The screenshot shows the SecureTask dashboard in a web browser. The browser's address bar displays "127.0.0.1:5000/dashboard". The page has a dark header with "SecureTask" on the left and navigation links "Dashboard", "Profile", and "Logout" on the right. The main content area is titled "Task Dashboard" and includes a "My Profile" button. Below the title is a search bar labeled "Search tasks by title..." and a dropdown menu for "All Categories" with a "Filter" button. A section titled "Add New Task" contains input fields for "Task Title", "Task Description", and a "Select Category (Optional)" dropdown, followed by a green "Create Task" button. At the bottom, the "Your Tasks" section shows a task titled "abcd" with a subtext "week 1 task <3".

2. LOGIN PAGE

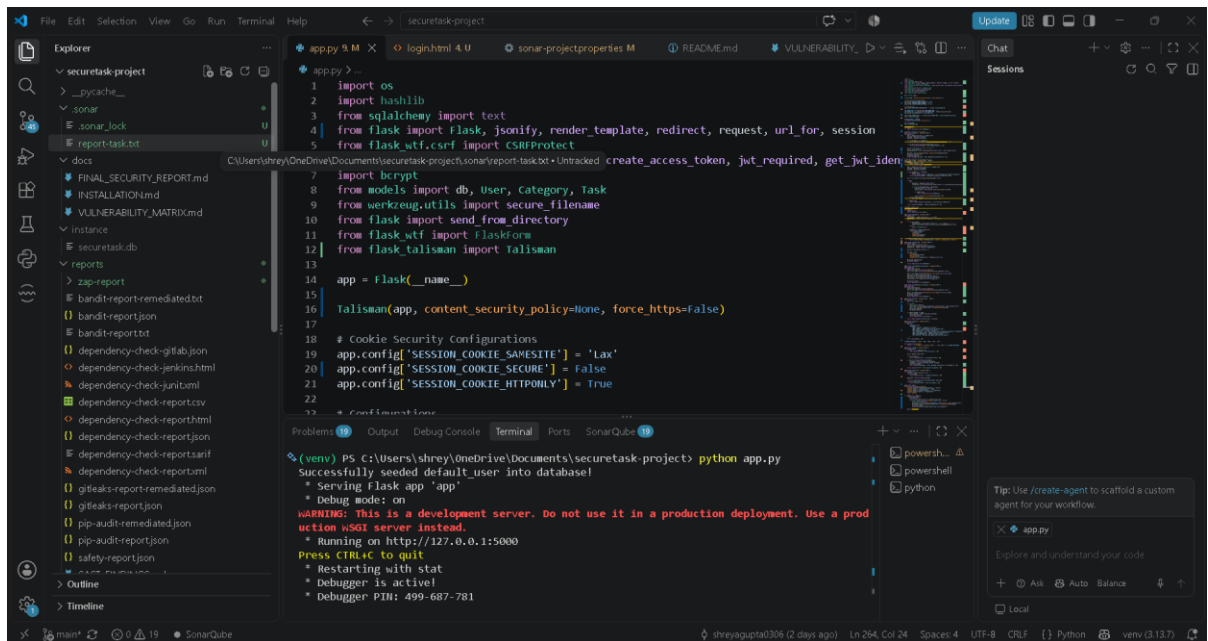


The screenshot shows the SecureTask login page in a web browser. The browser's address bar displays "127.0.0.1:5000/login". The page has a dark header with "SecureTask" on the left and navigation links "Dashboard", "Profile", and "Logout" on the right. The main content area features a white "Login" form with a shadow. The form includes a "Username" field with the text "default_user", a "Password" field with masked characters "*****", and a blue "Sign In" button.

3. USER PROFILE DETAILS



4. TERMINAL CODE/EDITOR



5. Database File Structure

